

Data Mining and Visualization

Final Project Report
Online Shopper Intent Classification

Submitted by

Noam Frei
Uri Lubetzky
Oren Turiel

Course: Data Mining and Visualization

Final Project

Submission Date: June 2026

Contents

1	Introduction	3
2	Getting to Know the Data	4
2.1	Data Loading and Initial Inspection	4
2.2	Feature Types and Missing Values	4
2.3	Descriptive Statistics of Numerical Features	6
2.4	Data Quality Assessment	7
2.4.1	Missing Values	7
2.4.2	Skewness of Numerical Features	7
2.4.3	Distributions of Numerical Features	8
2.4.4	Outlier Analysis	9
2.4.5	Categorical Feature Distributions	10
2.4.6	Target Variable Distribution	11
2.5	Bivariate Analysis and Main Insights	12
2.5.1	Seasonal Pattern in High-Intent Sessions	12
2.5.2	Page Value as a Strong Separator Between Classes	13
2.5.3	High-Intent Users Browse Products More Deeply	14
2.5.4	Visitor Type and High-Intent Rate	14
2.5.5	Special Day Score and Purchase Intent	15
2.5.6	Weekend Effect	15
2.6	Correlation Analysis	16
2.6.1	Highly Correlated Feature Pairs	17
2.7	Summary of Data Exploration	18
3	Preprocessing	19
3.1	Removing a Highly Correlated Feature: <code>bounce_rate</code>	19
3.2	Outlier Treatment for <code>engagement_score</code>	19
3.3	Missing Value Imputation	19
3.4	Removing a Redundant Feature: <code>num_product_pages</code>	20
3.5	Logarithmic Transformation of Skewed Features	20
3.6	Categorical Encoding	21
3.7	Feature Scaling	21
3.8	Final Preprocessed Training Set	22
3.9	Test Set Preprocessing	22
3.10	Preprocessing Summary	23
4	Classification	23
4.1	Loading the Preprocessed Data	23
4.2	Evaluation Metrics and Class Imbalance	23
4.3	Train-Validation Split	24
4.4	Candidate Models	24
4.4.1	Logistic Regression	25
4.4.2	Decision Tree	25
4.4.3	Support Vector Machine	25
4.4.4	LightGBM	25
4.4.5	CatBoost	25
4.5	10-Fold Stratified Cross-Validation	26
4.6	Validation Set Evaluation	26
4.7	Initial Model Comparison	26
4.8	Confusion Matrices	27
4.9	ROC Curve Analysis	27
4.10	Model Comparison Visualization	28
4.11	Model Behavior Discussion	29
4.11.1	Logistic Regression	29
4.11.2	Decision Tree	29
4.11.3	Support Vector Machine	29
4.11.4	LightGBM	29
4.11.5	CatBoost	29

4.12	Decision Tree Visualization and Feature Importance	29
4.13	Hyperparameter Tuning	31
4.14	Tuned Model Validation Results	31
4.15	Overfitting and Underfitting Analysis	32
4.16	Statistical Significance Testing	32
4.17	Imbalanced Data Handling	33
4.18	Cost-Sensitive Learning	34
4.19	Precision-Recall Curves	34
4.20	Final Individual Model Evaluation	35
4.21	Voting Ensemble and Final Model Selection	35
5	Final Summary	36

1 Introduction

This project focuses on analyzing user behavior in an online shopping platform and building a classification model that predicts whether a user session represents high purchase or engagement intent. Each row in the dataset represents a single browsing session, described by behavioral, contextual, and technical attributes such as the number of pages visited, time spent on different page types, bounce rate, exit rate, traffic source, visitor type, and engagement-related scores.

The project was divided into three main stages. First, we explored the training dataset in order to understand the structure of the data, the distribution of the features, the relationship between attributes, and the behavior of the target variable. Second, we applied preprocessing steps such as cleaning, handling irregular values, transforming features, and preparing the data for machine learning models. Finally, we trained and compared several classification models, tuned their hyperparameters, evaluated them using appropriate metrics, and selected the best-performing model for generating predictions on the provided test dataset.

The target variable in this project is `high_intent`, which indicates whether a session is classified as a high-intent session. Since high-intent sessions are important from a business point of view, the goal of the classification task is not only to achieve high accuracy, but also to correctly identify users who are likely to perform valuable actions on the website.

2 Getting to Know the Data

2.1 Data Loading and Initial Inspection

The first stage of the project was to load the training dataset and inspect its basic structure. The dataset contains user sessions from an online shopping platform, where each row represents one complete session. The goal is to predict the target variable `high_intent`, which indicates whether a session reflects a strong likelihood of meaningful user interaction or conversion.

After loading the file `shopper_train.csv`, we found that the training set contains:

- **9864 rows**, where each row represents one user session.
- **20 columns**, including behavioral features, contextual features, technical features, and the target variable.

The features include page visit counts, time spent on different page categories, engagement indicators, traffic information, visitor type, and session context. This initial inspection confirmed that the dataset contains both numerical and categorical variables, meaning that different preprocessing methods would be required before applying machine learning models.

	num_admin_pages	admin_duration	num_info_pages	info_duration	num_product_pages	product_duration	bounce
0	0	0.0	0	0.0	3	13.000000	0.00
1	0	0.0	0	0.0	1	0.000000	0.20
2	0	0.0	0	0.0	8	101.750000	0.00
3	0	0.0	2	38.2	121	3362.749679	0.00
4	2	2.0	0	0.0	0	NaN	0.10

Figure 1: First rows of the training dataset. Each row represents one user session, and each column describes a behavioral, contextual, or technical attribute of that session.

From the first rows of the dataset, we can already observe several important characteristics. Some sessions contain very little activity, such as sessions with zero administrative or informational pages. Other sessions contain stronger browsing behavior, for example a large number of product pages and long product duration. This shows that the dataset includes sessions with very different levels of user engagement, which is important for the classification task.

2.2 Feature Types and Missing Values

Next, we examined the data types, the number of missing values, the percentage of missing values, and the number of unique values in each column. This step is important because it helps identify which features are numerical, which features are categorical, and which features require cleaning before modeling.

	Data Type	Missing Values	Missing %	Unique Values
num_admin_pages	int64	0	0.00	26
admin_duration	float64	0	0.00	2805
num_info_pages	int64	0	0.00	16
info_duration	float64	0	0.00	1064
num_product_pages	int64	0	0.00	291
product_duration	float64	398	4.03	7480
bounce_rate	float64	1157	11.73	1466
exit_rate	float64	0	0.00	3974
page_value	float64	496	5.03	2082
special_day_score	float64	0	0.00	6
month	object	1460	14.80	10
operating_system	int64	0	0.00	8
browser	int64	0	0.00	12
region	int64	0	0.00	9
traffic_type	int64	0	0.00	20
visitor_type	object	996	10.10	3
is_weekend	int64	0	0.00	2
engagement_score	float64	0	0.00	8909
abandonment_score	float64	0	0.00	4463
high_intent	int64	0	0.00	2

Figure 2: Summary of feature data types, missing values, missing value percentages, and number of unique values.

Most of the numerical behavioral features, such as `num_admin_pages`, `num_info_pages`, `num_product_pages`, `admin_duration`, `info_duration`, `exit_rate`, `engagement_score`, and `abandonment_score`, are stored as integer or floating-point values. Several technical and contextual variables, such as `operating_system`, `browser`, `region`, and `traffic_type`, are represented as integer-coded categorical variables. In contrast, `month` and `visitor_type` are stored as object-type categorical variables.

The missing value analysis showed that most features do not contain missing values. However, several important columns do contain missing data:

- `product_duration`: 398 missing values, approximately 4.03%.
- `bounce_rate`: 1157 missing values, approximately 11.73%.
- `page_value`: 496 missing values, approximately 5.03%.
- `month`: 1460 missing values, approximately 14.80%.

- `visitor_type`: 996 missing values, approximately 10.10%.

These missing values are significant enough that they cannot simply be ignored. Therefore, they must be handled carefully during preprocessing. For numerical features, suitable imputation methods should be considered based on the distribution of each feature. For categorical features such as `month` and `visitor_type`, missing values should be treated in a way that preserves the meaning of the category and avoids introducing artificial numerical relationships.

2.3 Descriptive Statistics of Numerical Features

We then calculated descriptive statistics for the main numerical features. Integer-coded categorical variables such as `operating_system`, `browser`, `region`, and `traffic_type` were excluded from this table, because their numerical values represent category labels rather than continuous quantities.

	count	mean	std	min	25%	50%	75%	max
num_admin_pages	9864.0	2.3160	3.3178	0.0	0.0000	1.0000	4.0000	27.0000
admin_duration	9864.0	80.6495	176.6616	0.0	0.0000	7.0000	94.0000	3398.7500
num_info_pages	9864.0	0.5053	1.2801	0.0	0.0000	0.0000	0.0000	24.0000
info_duration	9864.0	34.8506	143.0302	0.0	0.0000	0.0000	0.0000	2549.3750
num_product_pages	9864.0	31.9672	45.2119	0.0	7.0000	18.0000	38.0000	705.0000
product_duration	9466.0	1210.7741	1995.7962	0.0	181.0000	601.7792	1475.9182	63973.5222
bounce_rate	8707.0	0.0227	0.0493	0.0	0.0000	0.0032	0.0171	0.2000
exit_rate	9864.0	0.0434	0.0490	0.0	0.0143	0.0253	0.0500	0.2000
page_value	9368.0	5.9088	18.3766	0.0	0.0000	0.0000	0.0000	361.7637
special_day_score	9864.0	0.0612	0.1985	0.0	0.0000	0.0000	0.0000	1.0000
engagement_score	9864.0	15.2683	18.6523	0.0	5.0755	9.7516	18.1499	291.0692
abandonment_score	9864.0	0.0329	0.0479	0.0	0.0080	0.0158	0.0329	0.2000

Figure 3: Descriptive statistics of the main numerical features in the training dataset. Integer-coded categorical variables were excluded from this summary.

The descriptive statistics reveal several important patterns. First, many page-count and duration-based features are highly skewed. For example, the median number of product pages is 18, while the maximum value is 705. Similarly, the median product duration is approximately 602 seconds, while the maximum value is above 63,000 seconds. This indicates that most users spend a moderate amount of time browsing, while a small number of sessions contain extremely large values.

A similar pattern appears in the duration features. For example, `admin_duration` and `info_duration` both have a median close to zero, but their maximum values are much higher. This suggests that many users do not visit administrative or informational pages at all, while a smaller group of users spends a significant amount of time on these pages.

The feature `page_value` is also highly skewed. Its median is zero, while its maximum value is above 361. This is meaningful because many sessions have no direct commercial value, while a smaller number of sessions are associated with high-value pages. Since `page_value` is likely related to conversion behavior, it may be an important predictor for `high_intent`.

The rate-based features, such as `bounce_rate`, `exit_rate`, and `abandonment_score`, are bounded between 0 and 0.2. These features describe disengagement behavior, so they are expected to be useful for separating low-intent sessions from high-intent sessions. In contrast, `engagement_score` has a much wider range, from 0 to approximately 291, showing that users differ greatly in their level of interaction with the website.

Overall, the initial statistical analysis suggests that preprocessing will need to handle missing values, skewed numerical distributions, categorical variables, and features with very different scales. “

“`latex`

2.4 Data Quality Assessment

Before choosing preprocessing methods, we performed a data quality assessment. The goal of this step was to identify missing values, irregular feature distributions, strong skewness, and possible outliers. This is important because classification models are sensitive to the way the data is represented, and poor handling of missing or extreme values may lead to unreliable results.

2.4.1 Missing Values

We first examined missing values at the column level. The analysis showed that missing values appear only in a small subset of the features, while most columns are complete.

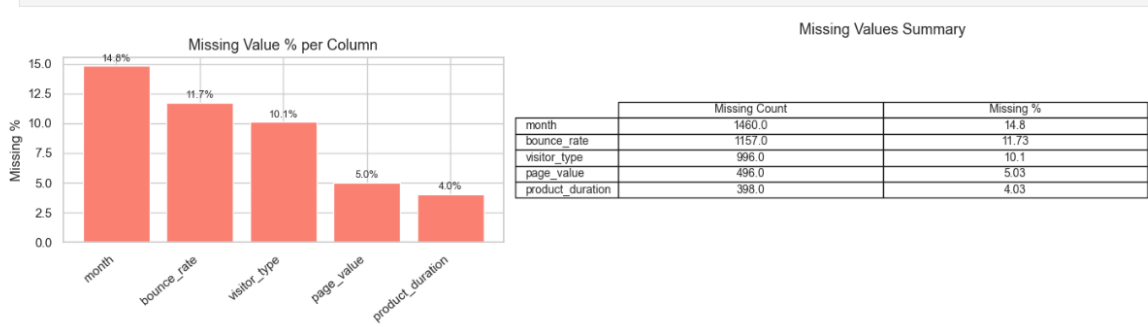


Figure 4: Missing value percentage and missing value count for each column containing missing data.

The feature with the highest missing percentage is **month**, with 1460 missing values, which is approximately 14.8% of the dataset. This is followed by **bounce_rate**, with 1157 missing values, or approximately 11.7%. The feature **visitor_type** contains 996 missing values, approximately 10.1%. The numerical features **page_value** and **product_duration** contain smaller missing percentages of approximately 5.0% and 4.0%, respectively.

This means that the missing values are not negligible, especially for **month**, **bounce_rate**, and **visitor_type**. Therefore, deleting all rows with missing values would remove a large amount of useful data. Instead, the missing values should be handled during preprocessing using appropriate imputation methods according to the feature type.

We also examined missing values at the row level. Most rows do not contain missing values, but a significant number of rows contain one or more missing entries.

Figure 5: Number of missing values per row in the training dataset.

The row-level analysis showed that 6056 rows have no missing values, while 3153 rows contain exactly one missing value. In addition, 611 rows contain two missing values, and 44 rows contain three missing values. Since most incomplete rows contain only one missing value, it is reasonable to preserve these rows and fill the missing entries instead of removing them.

2.4.2 Skewness of Numerical Features

Next, we examined the skewness of the numerical features. Skewness measures how asymmetric a distribution is. A value close to zero indicates a relatively balanced distribution, while a large positive value indicates that most observations are concentrated near small values, with a long tail toward large values.

	Feature	Skewness
0	product_duration	7.755150
1	info_duration	7.599689
2	page_value	6.313180
3	admin_duration	5.743064
4	num_product_pages	4.266238
5	num_info_pages	4.159568
6	engagement_score	4.084522
7	special_day_score	3.311363
8	bounce_rate	2.892041
9	abandonment_score	2.632243
10	exit_rate	2.127351
11	num_admin_pages	1.921935

Features with $|\text{skewness}| > 1$ are candidates for log transformation.

Figure 6: Skewness values of the numerical features. Features with high positive skewness are candidates for logarithmic transformation.

The skewness analysis shows that many numerical features are strongly right-skewed. The most skewed features are **product_duration**, **info_duration**, **page_value**, and **admin_duration**. These features describe time spent on pages or commercial value, so the skewness is expected: many users spend little or no time on certain page types, while a small number of users spend a much longer time.

The high skewness suggests that applying a logarithmic transformation may be useful for some features. A log transformation reduces the effect of extremely large values while preserving the order of the data. This can help models that are sensitive to scale and extreme values, such as KNN and logistic regression.

2.4.3 Distributions of Numerical Features

To better understand the numerical features, we plotted their raw distributions.

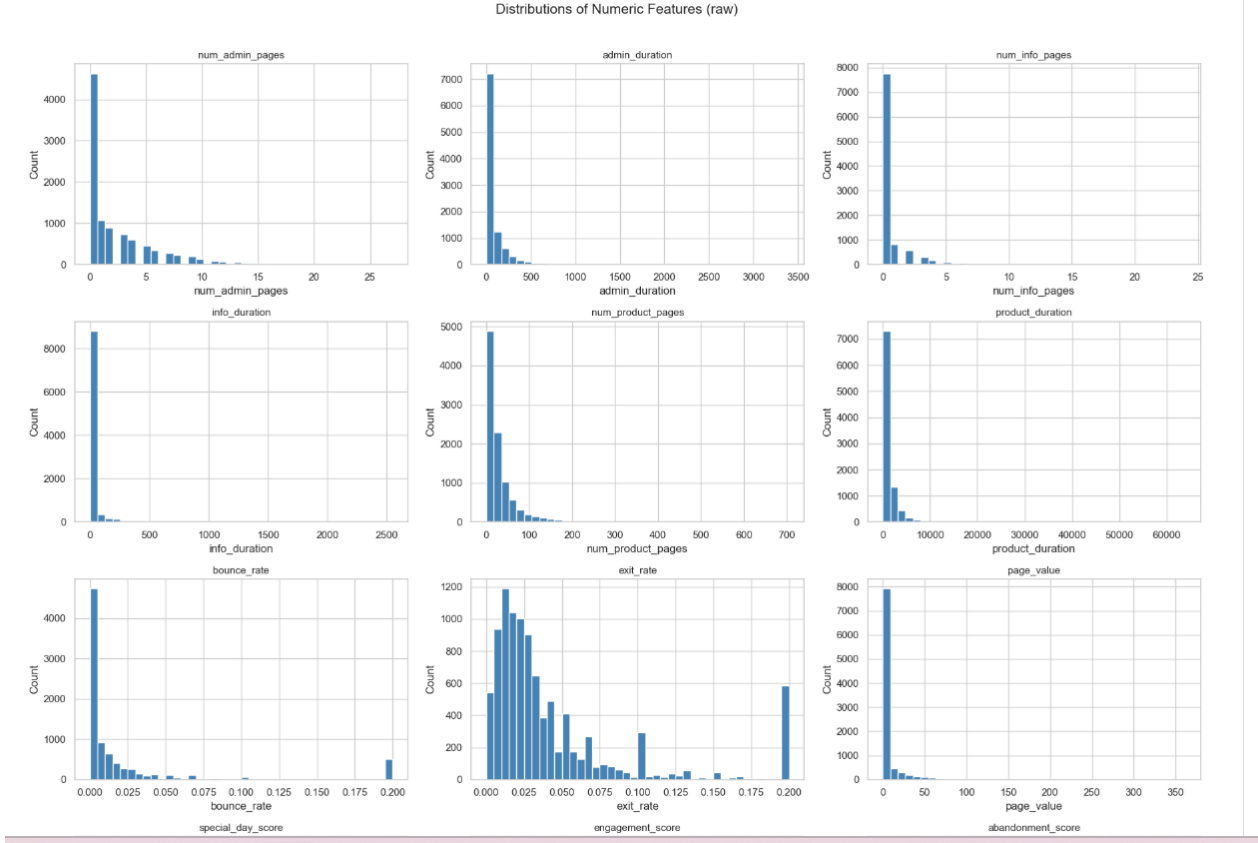


Figure 7: Raw distributions of the main numerical features. Many features are concentrated near zero and have long right tails.

The histograms confirm the skewness results. Several features, such as `admin_duration`, `info_duration`, `product_duration`, and `page_value`, have most of their values close to zero, with a small number of very large observations. This indicates that many sessions contain little activity in these categories, while a smaller number of sessions show much stronger browsing behavior.

The feature `num_product_pages` is also right-skewed. Most sessions contain a relatively small or moderate number of product page visits, but some sessions contain hundreds of product page views. This may represent users with high browsing activity, repeated product comparison, or unusual session behavior.

The rate-based variables, such as `bounce_rate`, `exit_rate`, and `abandonment_score`, are bounded and have different behavior from the duration features. These variables are important because they describe user disengagement. For example, high bounce or exit rates may indicate lower intent, while lower values may suggest that the user continued browsing.

2.4.4 Outlier Analysis

We also used boxplots to inspect potential outliers in the numerical features.

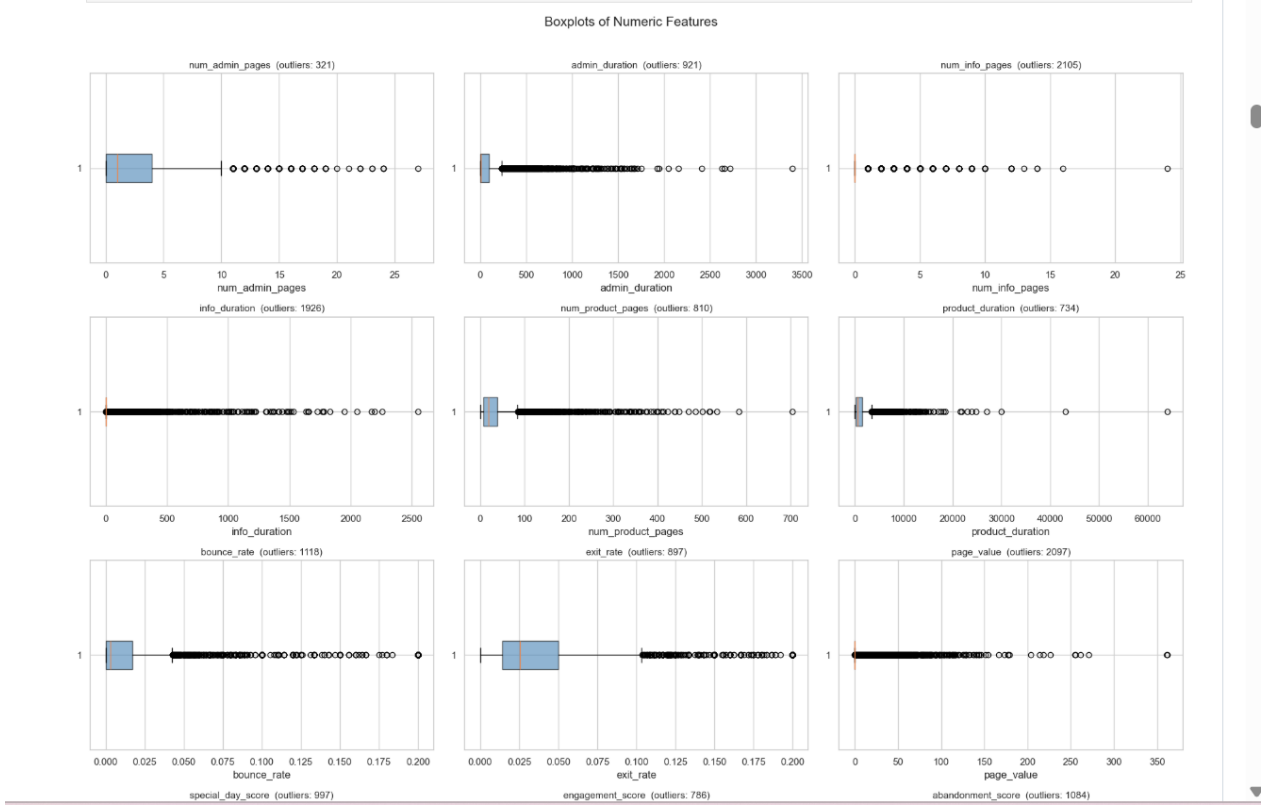


Figure 8: Boxplots of numerical features. The plots show many upper outliers, especially in duration, page count, and value-related features.

The boxplots show that many numerical features contain upper outliers. This is especially clear in `admin_duration`, `info_duration`, `product_duration`, `page_value`, and `num_product_pages`. However, these values should not automatically be treated as errors. In this dataset, large values may represent real user behavior, such as a user spending a long time browsing products or visiting many product pages during one session.

Therefore, instead of removing these observations immediately, we decided to handle them carefully during preprocessing. In particular, logarithmic transformations and scaling are more appropriate than simple deletion, because they reduce the influence of extreme values while preserving useful information.

2.4.5 Categorical Feature Distributions

We then examined the distributions of the categorical and integer-coded categorical features.

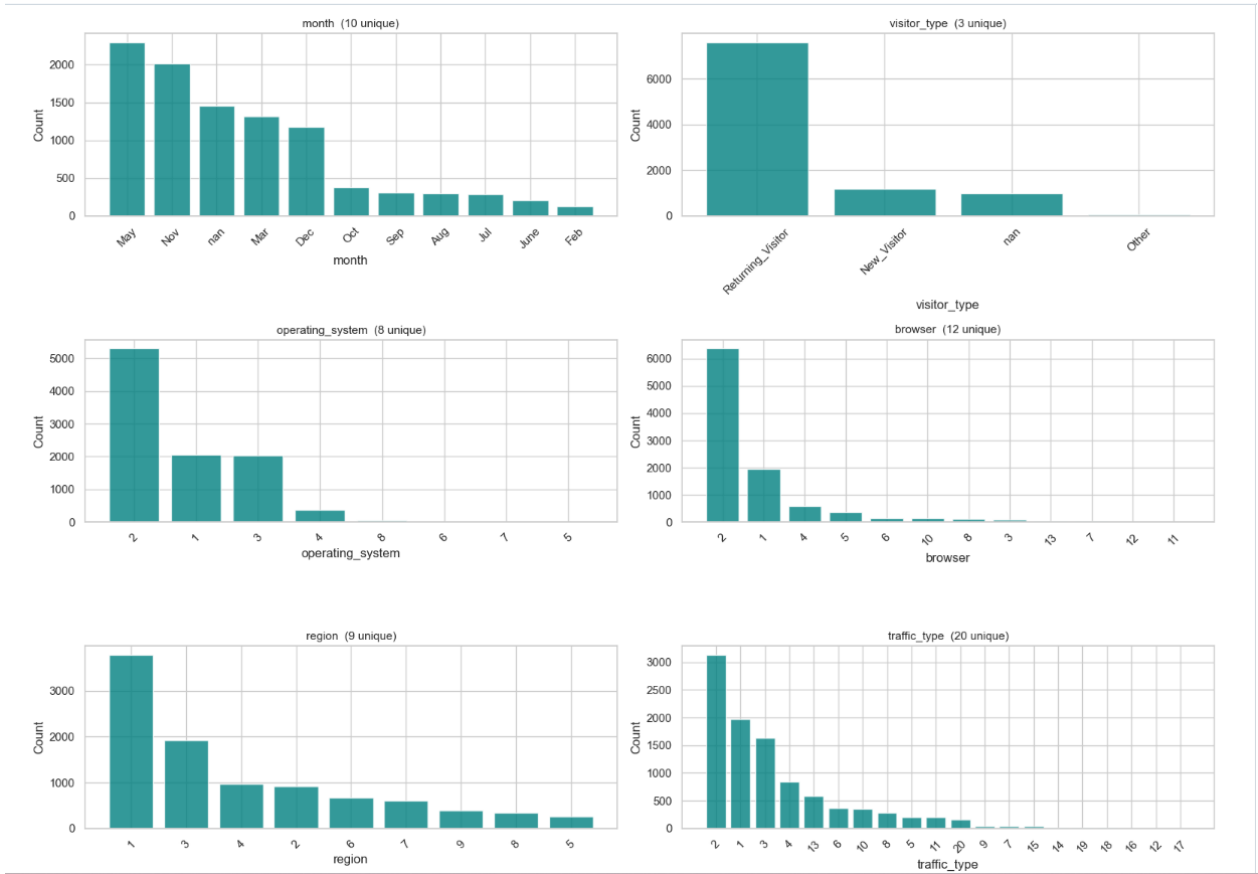


Figure 9: Distributions of categorical and integer-coded categorical features.

The categorical feature distributions are not uniform. For example, most sessions belong to a small number of months, with May and November appearing most frequently. The feature `visitor_type` is dominated by returning visitors, while new visitors form a much smaller group. Similarly, `operating_system`, `browser`, `region`, and `traffic_type` are concentrated around a few common categories.

This observation is important for preprocessing. Since these features represent categories rather than continuous numerical quantities, they should not be treated as regular numerical variables. For example, browser category “2” is not necessarily larger or better than browser category “1”. Therefore, categorical encoding is required before applying most machine learning models.

2.4.6 Target Variable Distribution

Finally, we examined the distribution of the target variable, `high_intent`.

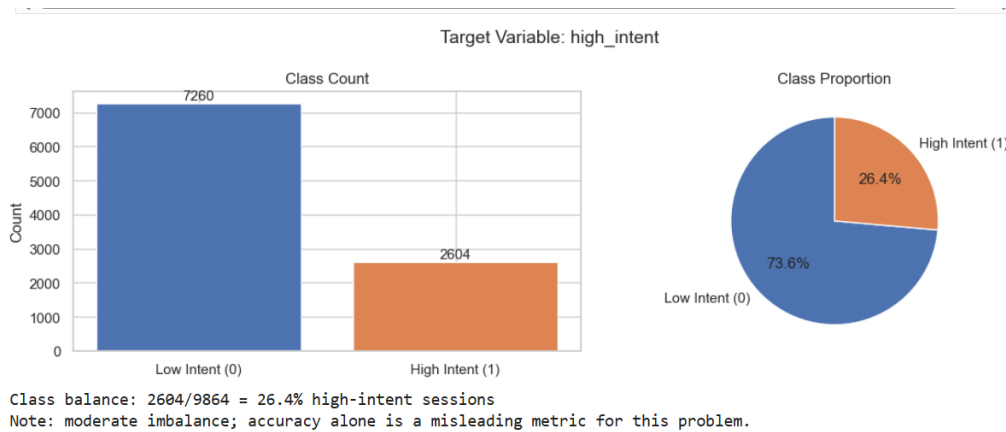


Figure 10: Distribution of the target variable `high_intent`.

The target distribution shows that the dataset is moderately imbalanced. Out of 9864 sessions, 7260 are labeled as low intent, while 2604 are labeled as high intent. This means that high-intent sessions represent approximately 26.4% of the data, while low-intent sessions represent approximately 73.6%.

This class imbalance is important for model evaluation. If we only use accuracy, a model may achieve a seemingly good score by mainly predicting the majority class. However, the business goal is to identify high-intent users, so evaluation metrics such as precision, recall, F1-score, and ROC-AUC are also needed. This observation directly affects the classification stage, where we compare models using metrics that better reflect performance on both classes.

2.5 Bivariate Analysis and Main Insights

After examining each feature separately, we analyzed relationships between the features and the target variable. This stage is important because it helps us understand which attributes may be useful for predicting `high_intent`. We focused on relationships that provide meaningful, non-trivial insights about user behavior rather than only showing basic distributions.

2.5.1 Seasonal Pattern in High-Intent Sessions

The first important pattern is related to the month of the session. The high-intent rate is not uniform across the year, which suggests that the time of year contains useful information for predicting user intent.



Figure 11: High-intent rate and session volume by month.

The results show a clear seasonal pattern. February has the lowest high-intent rate, approximately 3.8%, while November has the highest high-intent rate, approximately 38.5%. Other months such as September and October also show relatively high intent rates of around 31%.

This suggests that user intent changes strongly throughout the year. A likely explanation is that online shopping behavior is influenced by seasonal events, holidays, promotions, and end-of-year shopping periods. November is especially important because it combines both a high number of sessions and a high high-intent rate. Therefore, `month` appears to be a meaningful contextual feature for the classification model.

month		
Feb	0.038462	130
Mar	0.159939	1313
May	0.219948	2296
June	0.229268	205
Jul	0.274306	288
Aug	0.290541	296
Sep	0.310680	309
Oct	0.306069	379
Nov	0.385417	2016
Dec	0.231229	1172

Figure 12: High-intent rate and number of sessions for each month.

The table supports the same conclusion. For example, May has the largest session volume, but its high-intent rate is lower than November. This means that high traffic volume does not necessarily imply high purchase intent. Therefore, the model should not only learn how many sessions appear in each month, but also the relationship between month and conversion tendency.

2.5.2 Page Value as a Strong Separator Between Classes

The second major insight is that `page_value` separates low-intent and high-intent sessions very clearly. This feature represents the estimated commercial value of the pages visited during the session, and it appears to be highly related to the target variable.

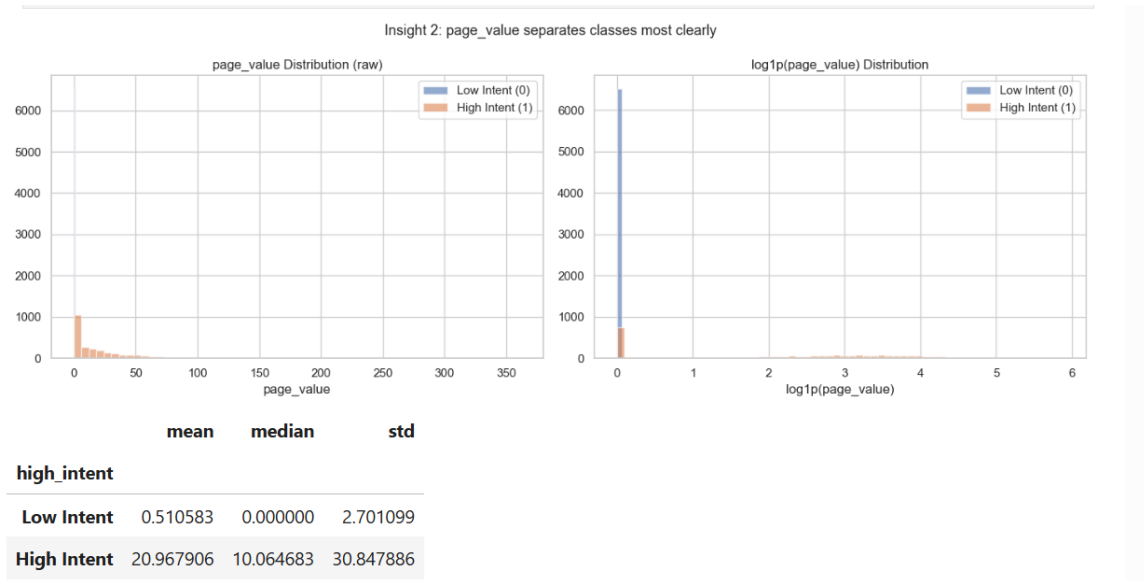


Figure 13: Distribution of `page_value` by class, shown in raw scale and after logarithmic transformation.

The raw distribution shows that most low-intent sessions have a `page_value` close to zero. In contrast, high-intent sessions are much more likely to have positive and larger `page_value` values. Because the raw

feature is highly skewed, the logarithmic version, $\log_{1p}(\text{page_value})$, gives a clearer view of the separation between the classes.

The summary statistics confirm this difference. Low-intent sessions have an average `page_value` of approximately 0.51 and a median of 0. In contrast, high-intent sessions have an average `page_value` of approximately 20.97 and a median of approximately 10.06. This is a large difference, indicating that `page_value` is one of the strongest predictors in the dataset.

From a business point of view, this result is reasonable. Users who visit pages with higher commercial value are more likely to show meaningful engagement or conversion intent. Therefore, this feature should be preserved and transformed carefully during preprocessing.

2.5.3 High-Intent Users Browse Products More Deeply

The third insight is that high-intent users tend to spend more time browsing products and show higher engagement scores.

Figure 14: Distributions of product duration and engagement score by class after logarithmic transformation.

The distributions show that high-intent sessions are shifted to the right compared to low-intent sessions. This means that high-intent users generally spend more time on product pages and have higher engagement scores. The class averages support this observation: low-intent sessions have an average `product_duration` of approximately 714.09, while high-intent sessions have an average of approximately 2596.25. Similarly, the average `engagement_score` increases from approximately 9.71 for low-intent sessions to approximately 30.77 for high-intent sessions.

This pattern is important because it reflects real browsing behavior. Users who are more interested in products usually spend more time comparing items, reading product information, and interacting with the website. Therefore, both `product_duration` and `engagement_score` are expected to be useful features for predicting `high_intent`.

2.5.4 Visitor Type and High-Intent Rate

We also examined whether the type of visitor is related to high-intent behavior.

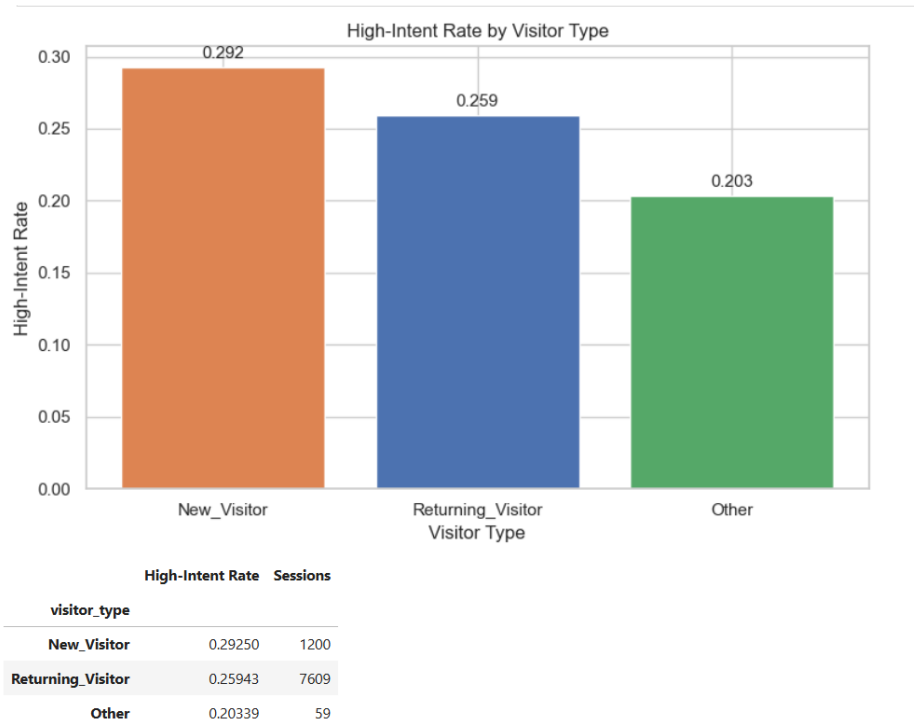


Figure 15: High-intent rate by visitor type.

The results show that new visitors have the highest high-intent rate, approximately 29.3%. Returning visitors have a slightly lower rate of approximately 25.9%, while the `Other` category has the lowest rate,

approximately 20.3%. However, the number of sessions in the **Other** category is very small compared to the other groups, so conclusions about this category should be treated carefully.

This result is interesting because returning visitors might be expected to have higher intent, but the data shows that new visitors are slightly more likely to be labeled as high intent. This may indicate that some new users arrive through targeted campaigns, product searches, or direct purchase-oriented traffic. Since visitor type provides useful behavioral context, it should be encoded and included in the models.

2.5.5 Special Day Score and Purchase Intent

Next, we examined the relationship between `special_day_score` and the high-intent rate.

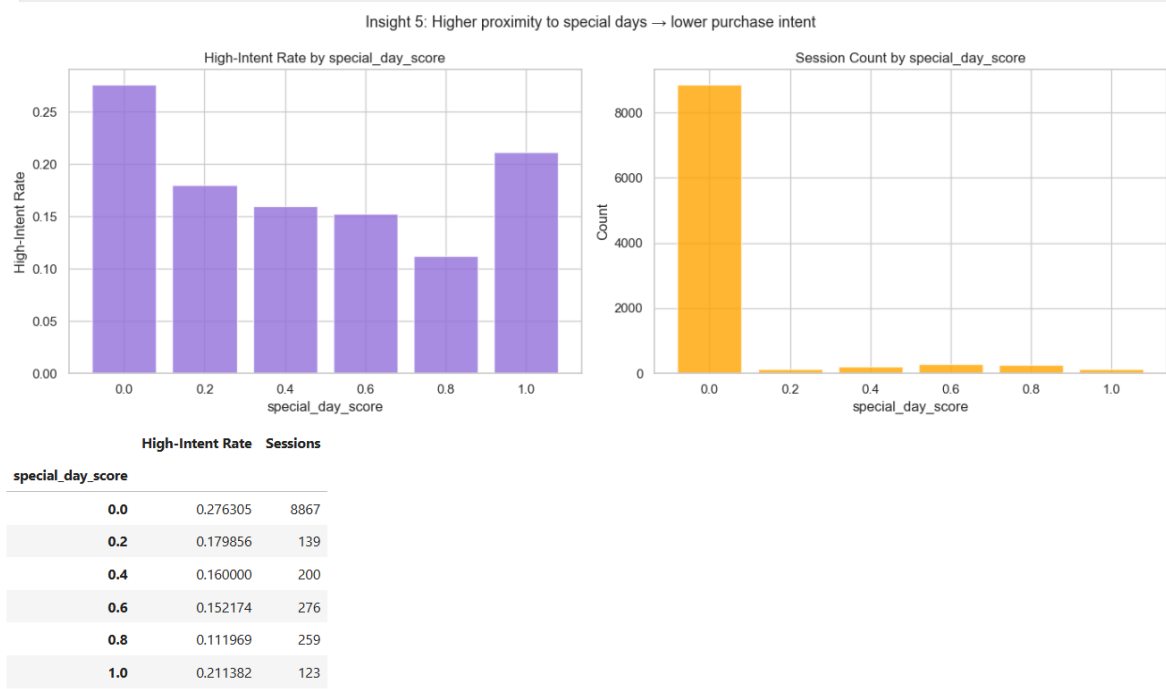


Figure 16: High-intent rate and session count by `special_day_score`.

The results show that sessions with `special_day_score` = 0 have the highest high-intent rate, approximately 27.6%. For intermediate values such as 0.2, 0.4, 0.6, and 0.8, the high-intent rate is lower. At `special_day_score` = 1.0, the rate increases again to approximately 21.1%, but it is still lower than the rate for score 0.

This means that proximity to special days does not automatically increase high-intent behavior in this dataset. One possible explanation is that many users browse before special days without immediately converting. Another important point is that most sessions have `special_day_score` = 0, while the other values have much smaller sample sizes. Therefore, this feature may still be useful, but the interpretation should be careful because the groups are not equally represented.

2.5.6 Weekend Effect

Finally, we compared high-intent rates between weekday and weekend sessions.

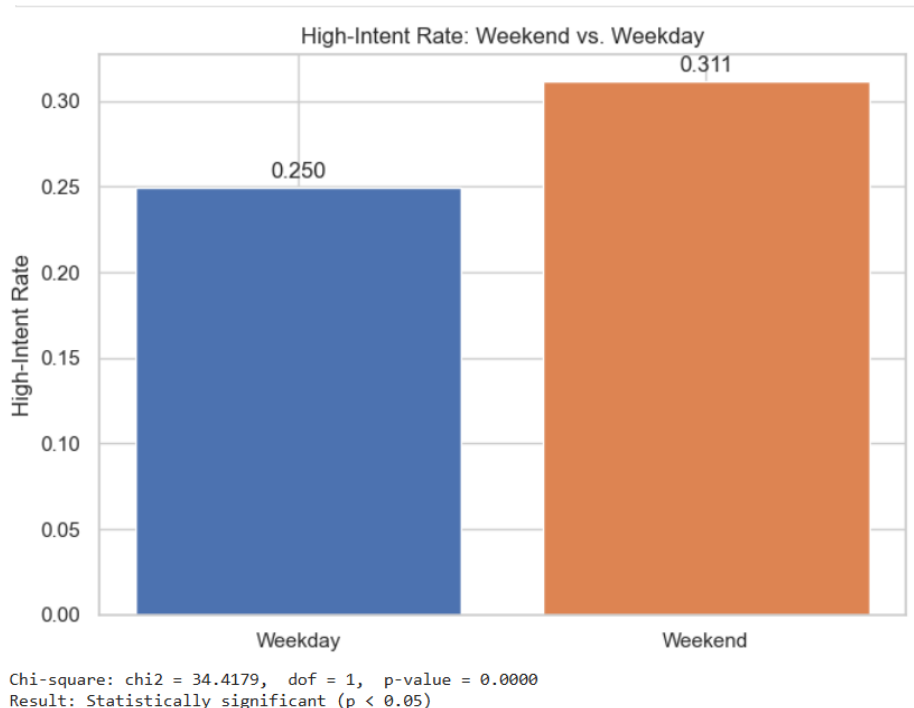


Figure 17: High-intent rate for weekday and weekend sessions.

The high-intent rate is higher during weekends. Weekday sessions have a high-intent rate of approximately 25.0%, while weekend sessions have a rate of approximately 31.1%. A chi-square test was also performed, and the result was statistically significant with a p-value below 0.05.

This suggests that weekend browsing behavior is meaningfully different from weekday behavior. Users may have more free time during weekends, allowing them to browse products more deeply and make more purchase-related decisions. Therefore, `is_weekend` should be included as a predictive feature in the classification models.

2.6 Correlation Analysis

As a final step before preprocessing, we examined the pairwise Pearson correlations between the numerical features. The goal of this analysis was to identify strong relationships between features, detect possible redundancy, and understand which numerical variables are most directly related to the target variable `high_intent`.

For the correlation matrix, integer-coded categorical variables such as `operating_system`, `browser`, `region`, and `traffic_type` were excluded. Although these columns are stored numerically, their values represent categories and therefore Pearson correlation is not meaningful for them in the same way it is for continuous numerical features.

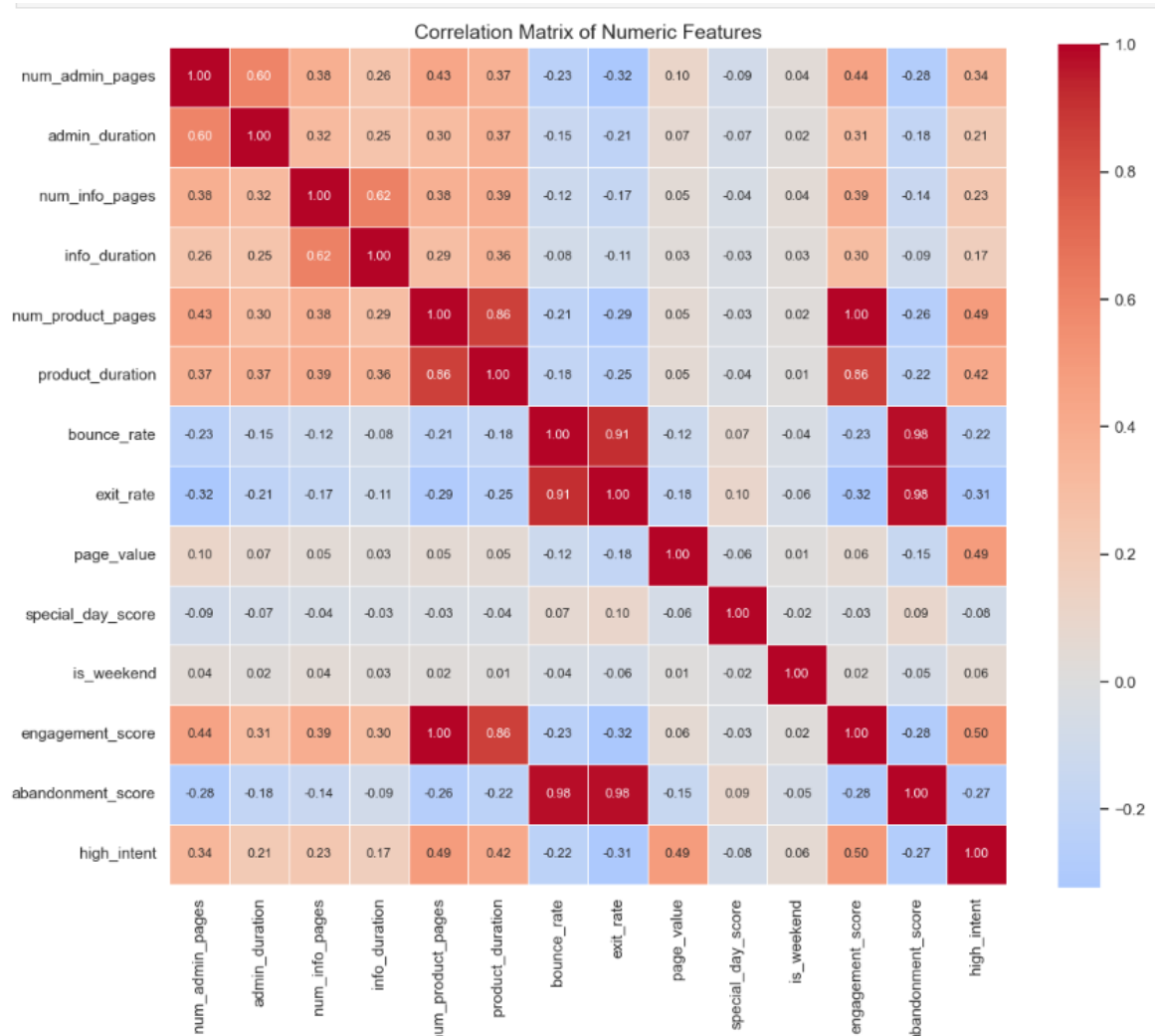


Figure 18: Pearson correlation matrix of the main numerical features. Integer-coded categorical variables were excluded.

The correlation matrix shows several important relationships. First, the features most positively correlated with `high_intent` are `engagement_score`, `page_value`, `num_product_pages`, and `product_duration`. Their correlations with the target are approximately 0.50, 0.49, 0.49, and 0.42, respectively. This supports the previous observations that high-intent users tend to browse more product pages, spend more time on products, and interact more deeply with the website.

On the other hand, several disengagement-related features are negatively correlated with `high_intent`. For example, `exit_rate` has a correlation of approximately -0.31 with the target, and `abandonment_score` has a correlation of approximately -0.27 . This means that sessions with stronger signs of abandonment are less likely to be high-intent sessions. This result is consistent with the meaning of these features.

The matrix also reveals strong correlations between some input features. For example, `num_product_pages` and `engagement_score` are almost perfectly correlated. This suggests that `engagement_score` is strongly based on product browsing activity, or at least that both features describe almost the same behavior. Similarly, `bounce_rate`, `exit_rate`, and `abandonment_score` are strongly correlated with each other, meaning that they capture related forms of user disengagement.

2.6.1 Highly Correlated Feature Pairs

To better understand the strongest relationships, we plotted several highly correlated feature pairs.

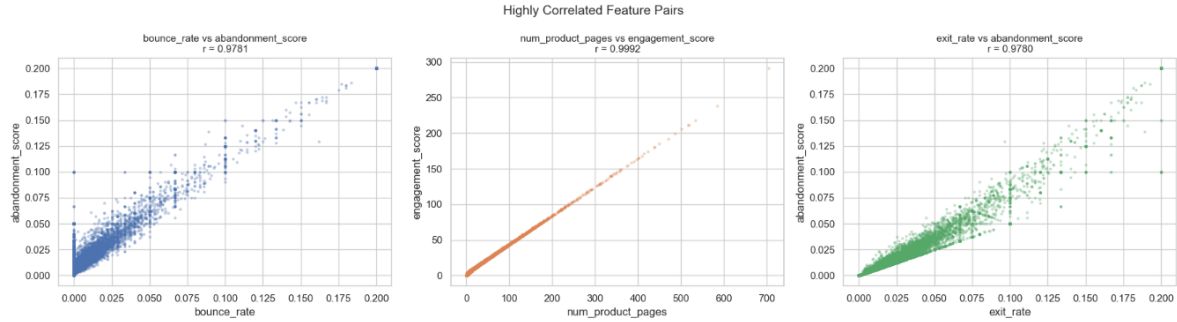


Figure 19: Scatter plots of highly correlated feature pairs.

The first plot shows a strong positive relationship between `bounce_rate` and `abandonment_score`, with a correlation of approximately 0.978. This is expected because both features describe disengagement from the session. As the bounce rate increases, the abandonment score also tends to increase.

The second plot shows an almost perfect relationship between `num_product_pages` and `engagement_score`, with a correlation of approximately 0.999. This means that these two features contain nearly the same information. Keeping both may still be useful for some models, such as tree-based models, but for models sensitive to redundant inputs, this could increase unnecessary complexity.

The third plot shows a strong positive relationship between `exit_rate` and `abandonment_score`, with a correlation of approximately 0.978. This again confirms that the abandonment-related variables are closely connected.

These results are important for preprocessing and model selection. Strongly correlated features are not necessarily wrong, but they can affect models differently. For example, distance-based models such as KNN may be influenced by redundant features because duplicated information can receive too much weight in the distance calculation. Linear models may also be affected by multicollinearity. In contrast, tree-based models such as Random Forests are usually more robust to correlated features, although redundancy can still make interpretation less clear.

2.7 Summary of Data Exploration

The exploratory analysis gave us several important conclusions that guided the preprocessing and classification stages.

First, the dataset contains 9864 training examples and 20 columns, including both numerical and categorical variables. The target variable is `high_intent`, where approximately 26.4% of the sessions are high-intent and 73.6% are low-intent. This means that the dataset is moderately imbalanced. Therefore, accuracy alone is not sufficient for model evaluation, because a model could obtain a reasonable accuracy by mainly predicting the majority class. For this reason, later evaluation should also include metrics such as precision, recall, F1-score, and ROC-AUC.

Second, the data quality assessment showed that several features contain missing values. The most affected columns are `month`, `bounce_rate`, `visitor_type`, `page_value`, and `product_duration`. Since many rows contain only one missing value, deleting all incomplete rows would remove useful information. Therefore, imputation is a better strategy than row deletion.

Third, many numerical features are strongly right-skewed. This is especially true for duration-based variables and `page_value`. The histograms and boxplots showed that many sessions have values close to zero, while a small number of sessions have very large values. These large values are not necessarily errors, since they may represent real users who spent a long time browsing or visited many product pages. Therefore, instead of removing them automatically, we decided that transformations such as `log1p` and scaling would be more appropriate.

Fourth, several features showed meaningful relationships with the target variable. `page_value` was one of the clearest separators between low-intent and high-intent sessions. High-intent users also tended to have larger `product_duration` and higher `engagement_score`. In addition, contextual variables such as `month`, `visitor_type`, and `is_weekend` showed useful patterns. For example, November had the highest high-intent rate, and weekend sessions had a significantly higher high-intent rate than weekday sessions.

Finally, the correlation analysis showed that some features are highly redundant. In particular, `num_product_pages` and `engagement_score` are almost perfectly correlated, and the disengagement features `bounce_rate`, `exit_rate`, and `abandonment_score` are also strongly related. This does not mean that these features must automatically be removed, but it should be considered during preprocessing and model interpretation.

Based on these observations, the preprocessing stage should include handling missing values, encoding categorical variables, applying logarithmic transformations to skewed numerical features, scaling numerical features where needed, and carefully considering the effect of correlated features. These steps prepare the data for reliable classification models. “

3 Preprocessing

After completing the exploratory analysis, we prepared the dataset for classification. The preprocessing stage was based only on the training dataset, and all learned parameters were saved so that the exact same transformations could later be applied to the test dataset. This is important because the test set must be processed in the same way as the training set, without using information from the target variable or from future data.

The preprocessing included handling redundant features, treating outliers, imputing missing values, transforming skewed numerical variables, encoding categorical variables, scaling numerical features, and preparing the final training and test matrices.

3.1 Removing a Highly Correlated Feature: `bounce_rate`

Based on the correlation analysis, `bounce_rate` was found to be almost perfectly correlated with `abandonment_score`, with a correlation of approximately $r = 0.98$. In addition, `bounce_rate` had a relatively high missing value percentage of approximately 11.7%.

Since both variables describe similar disengagement behavior, keeping both would add redundant information to the model. It would also make the missing value imputation process more complicated, because `bounce_rate` contains many missing values. Therefore, we removed `bounce_rate` and kept `abandonment_score`, which provided a similar signal with no missing values.

After removing `bounce_rate`, the dataset changed from 20 columns to 19 columns.

3.2 Outlier Treatment for `engagement_score`

The feature `engagement_score` had a long right tail with several extreme values. These values may represent real user behavior, but they can still have a strong influence on models that are sensitive to scale and distance. Therefore, instead of deleting these observations, we clipped `engagement_score` at the 95th percentile.

The clipping threshold was:

$$\text{engagement_score}_{95\%} = 48.1141$$

All values above this threshold were replaced by the threshold value. In total, 494 values were clipped.

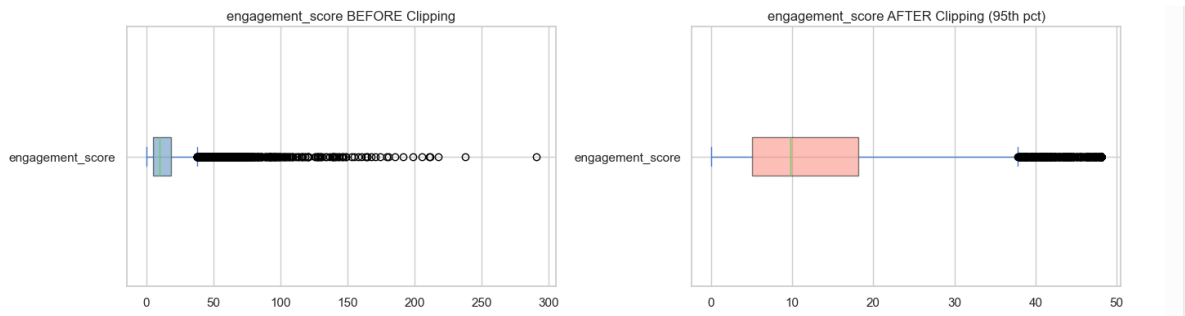


Figure 20: Boxplots of `engagement_score` before and after clipping at the 95th percentile.

The plot shows that before clipping, `engagement_score` had many very large values, reaching close to 300. After clipping, the upper range was limited while the main body of the distribution was preserved. This reduces the effect of extreme values without removing entire user sessions from the dataset.

3.3 Missing Value Imputation

The dataset contained missing values in several important columns: `product_duration`, `month`, `visitor_type`, and `page_value`. Since many rows had only one missing value, deleting incomplete rows would remove useful training data. Therefore, we used imputation rather than row deletion.

The imputation strategy was chosen separately for each feature, according to the meaning of the feature and the results of the exploratory analysis. The target variable `high_intent` was not used during the imputation process in order to avoid data leakage.

Feature	Missing Percentage	Imputation Strategy
<code>product_duration</code>	Approximately 4%	If <code>num_product_pages</code> = 0, missing values were filled with 0, because no product pages means no product browsing time. Remaining values were imputed using a predictive model.
<code>month</code>	Approximately 14%	Missing months were imputed using a classifier based on behavioral features.
<code>visitor_type</code>	Approximately 10%	Missing visitor types were imputed using a classifier based on behavioral features.
<code>page_value</code>	Approximately 5%	Missing values were imputed using a conditional median grouped by <code>traffic_type</code> .

Table 1: Summary of missing value imputation strategies.

For `product_duration`, the first imputation rule was deterministic. If a session had zero product pages and missing product duration, then the missing duration was replaced with zero. This is logical because a user cannot spend time on product pages if no product pages were visited. This rule filled 3 rows, leaving 395 missing values in `product_duration` to be handled by the remaining imputation method.

For categorical variables such as `month` and `visitor_type`, predictive imputation was used because these features are not continuous numerical quantities. For `page_value`, a grouped conditional median was used because the feature is highly skewed, and the median is more robust to extreme values than the mean.

3.4 Removing a Redundant Feature: `num_product_pages`

After the missing value imputation stage, we removed `num_product_pages`. This feature had an almost perfect correlation with `engagement_score`, with a correlation close to 1.00, as seen in figure 19.

We kept `num_product_pages` during the earlier preprocessing steps because it was useful for imputation. For example, it helped identify cases where `product_duration` should be zero, and it also provided useful behavioral information for predicting missing values. However, after imputation was completed, keeping both `num_product_pages` and `engagement_score` would introduce unnecessary redundancy.

After dropping `num_product_pages`, the dataset contained 18 columns, including the target variable.

3.5 Logarithmic Transformation of Skewed Features

The exploratory analysis showed that several numerical features were strongly right-skewed. This was especially true for the duration features and `page_value`. To reduce skewness and limit the influence of very large values, we applied a logarithmic transformation using:

$$x_{\text{new}} = \log(1 + x)$$

The `log1p` transformation was used instead of the regular logarithm because many observations are equal to zero. Since $\log(0)$ is undefined, using $\log(1 + x)$ allows zero values to remain valid.

The transformed features were:

`admin_duration`, `info_duration`, `product_duration`, `page_value`

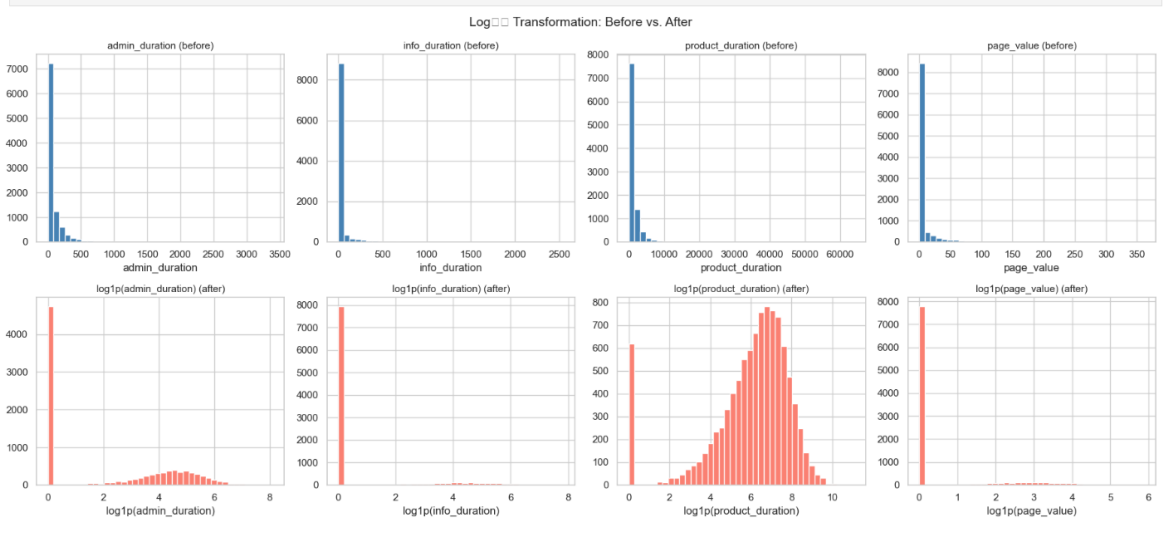


Figure 21: Distributions of selected numerical features before and after applying the $\log_{1p}$ transformation.

The figure shows that the logarithmic transformation reduced the effect of extreme values and made the distributions more compact. This is especially visible for `product_duration`, where the raw values had a very long right tail, while the transformed values became more balanced and easier for models to handle.

3.6 Categorical Encoding

The dataset contains several categorical features:

`month`, `visitor_type`, `operating_system`, `browser`, `region`, `traffic_type`

These variables cannot be used directly by most machine learning models in their original form. Therefore, we encoded each categorical column into integer labels.

Column	Unique Values	Encoded Range
<code>month</code>	10	0–9
<code>visitor_type</code>	3	0–2
<code>operating_system</code>	8	0–7
<code>browser</code>	12	0–11
<code>region</code>	9	0–8
<code>traffic_type</code>	20	0–19

Table 2: Categorical variables after label encoding.

Label encoding was chosen because the later classification stage mainly uses tree-based models, such as decision trees, random forests, and gradient boosting models. These models can handle integer-coded categorical variables better than distance-based models, and label encoding avoids creating a very large number of additional columns. This keeps the dataset compact and avoids unnecessary column expansion.

After label encoding, the dataset still contained 18 columns, because each categorical feature remained a single column.

3.7 Feature Scaling

After transformation and encoding, we scaled the main numerical features to the range $[0, 1]$ using Min-Max scaling. The scaling formula is:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

The scaled features were:

```
num_admin_pages, num_info_pages, admin_duration, info_duration,
product_duration, page_value, exit_rate, abandonment_score,
engagement_score, special_day_score
```

Scaling is important because some models are sensitive to the magnitude of the features. For example, KNN uses distances between samples, so features with larger numerical ranges may dominate the distance calculation if scaling is not applied. Scaling also makes the feature space more balanced and easier to compare.

After scaling, all selected numerical features were mapped to the range $[0, 1]$.

3.8 Final Preprocessed Training Set

At the end of the preprocessing stage, the target variable `high_intent` was separated from the input features. The final preprocessed training set had the following shape:

$$X_{\text{train}} : 9864 \times 17$$

$$y_{\text{train}} : 9864$$

There were no missing values left in the final training matrix. The class distribution remained unchanged:

$$\text{Low Intent} = 7260$$

$$\text{High Intent} = 2604$$

This means that preprocessing cleaned and transformed the input features without changing the target labels or altering the original class balance.

All learned preprocessing parameters were stored, including the clipping threshold, imputation models, encoders, scaler, and the lists of transformed columns. These stored parameters were later reused to preprocess the test set in exactly the same way.

3.9 Test Set Preprocessing

The test dataset was processed using the same pipeline learned from the training data. This is important because the test set must represent unseen data. Therefore, no preprocessing decision was learned directly from the test set, and the test set was not allowed to influence the training process.

The test set contained 2466 rows and 19 columns. It did not contain the target column `high_intent`, because the goal was to generate predictions for these sessions.

The missing values in the test set appeared in the same types of features as in the training set:

- `product_duration`: 95 missing values.
- `bounce_rate`: 322 missing values.
- `page_value`: 120 missing values.
- `month`: 389 missing values.
- `visitor_type`: 237 missing values.

The same transformations were then applied to the test set:

1. Drop `bounce_rate`.
2. Clip `engagement_score` using the 95th percentile threshold learned from the training set.
3. Impute missing values using the same rules and models learned from the training data.
4. Drop `num_product_pages`.
5. Apply the same `log1p` transformations.
6. Apply the same label encoders.
7. Apply the same Min-Max scaler.

This ensured that the final test matrix had the same structure and feature representation as the training matrix. Therefore, the trained classification models could be applied to the test data without feature mismatch or data leakage.

3.10 Preprocessing Summary

The preprocessing stage transformed the raw dataset into a clean and consistent format suitable for machine learning. Missing values were handled without deleting large parts of the dataset, skewed numerical variables were transformed, extreme engagement values were clipped, redundant highly correlated features were removed, categorical variables were encoded, and numerical variables were scaled.

The most important design choice in this stage was to avoid data leakage. All preprocessing decisions were learned only from the training dataset, and the exact same stored parameters were reused for the test dataset. This makes the model evaluation and final prediction process more reliable. “

4 Classification

After completing the preprocessing stage, we used the cleaned training data to train and compare several classification models. The goal of this stage was to predict the binary target variable `high_intent`, where class 1 represents a high-intent user session and class 0 represents a low-intent session.

The classification stage included loading the preprocessed datasets, choosing appropriate evaluation metrics, splitting the data into training and validation sets, defining several candidate models, evaluating them using stratified cross-validation, and comparing their validation performance.

4.1 Loading the Preprocessed Data

The classification stage used the CSV files produced at the end of preprocessing:

- `X_train_preprocessed.csv`: preprocessed training features.
- `y_train.csv`: training labels.
- `X_test_preprocessed.csv`: preprocessed test features.

After loading the files, the training feature matrix contained 9864 samples and 17 features:

$$X_{\text{train}} \in R^{9864 \times 17}$$

The training label vector contained 9864 target labels:

$$y_{\text{train}} \in R^{9864}$$

The test feature matrix contained 2466 samples and 17 features:

$$X_{\text{test}} \in R^{2466 \times 17}$$

This confirmed that the training and test datasets had the same feature structure after preprocessing. The final feature list included page-related variables, duration variables, contextual variables, categorical variables after encoding, and engagement-related features.

4.2 Evaluation Metrics and Class Imbalance

Before training the models, we examined the class distribution again. The dataset is moderately imbalanced:

$$\text{Low Intent (0)} = 7260$$

$$\text{High Intent (1)} = 2604$$

Therefore, the positive class rate is:

$$\frac{2604}{9864} \approx 26.4\%$$

This means that approximately 73.6% of the sessions are low-intent sessions, while only 26.4% are high-intent sessions.

Figure 22: Class distribution of the target variable `high_intent`.

Because of this imbalance, accuracy alone is not a sufficient evaluation metric. A simple classifier that always predicts the majority class, low intent, would already achieve approximately 73.6% accuracy, while completely failing to identify high-intent users. This would be misleading, especially because the business goal is to detect users who are likely to perform valuable actions.

For this reason, we used several evaluation metrics:

Metric	Reason for Use
Accuracy	Measures the overall proportion of correct predictions, but is not sufficient alone because the classes are imbalanced.
Precision	Measures how many predicted high-intent sessions are truly high intent. Low precision means that marketing resources may be wasted on users who are unlikely to convert.
Recall	Measures how many actual high-intent sessions were correctly detected. Low recall means that many valuable users are missed.
F1-score	Combines precision and recall into one balanced metric. This is useful when both false positives and false negatives matter.
ROC-AUC	Measures the model’s ability to separate the two classes across different classification thresholds.
PR-AUC	Focuses more directly on positive-class performance, which is useful when the positive class is less frequent.

Table 3: Evaluation metrics used for the classification task.

In this problem, false negatives are especially important. A false negative means that a real high-intent user is predicted as low intent, so the business may miss a potential conversion. False positives are also relevant, because they may cause marketing resources to be spent on low-intent users, but this is usually less costly than missing a valuable user completely. Therefore, we focused mainly on F1-score, recall, and ROC-AUC, while still reporting accuracy and precision.

4.3 Train-Validation Split

In addition to cross-validation, we created a separate validation set in order to compare model performance on unseen data. The training dataset was split into:

80% training data

20% validation data

The split was performed using stratification, meaning that the same class ratio was preserved in both the training and validation sets. This is important because the dataset is imbalanced. Without stratification, one split could accidentally contain a different proportion of high-intent sessions, which would make the validation results less reliable.

The resulting split was:

$$X_{\text{train split}} : 7891 \times 17$$

$$X_{\text{val}} : 1973 \times 17$$

Both sets kept the same positive class rate of approximately 26.4%.

4.4 Candidate Models

We trained and compared five classification models. The selected models cover several different modeling approaches, from simple linear classification to more advanced ensemble methods.

Model	Type	Preprocessing Used
Logistic Regression	Linear probabilistic classifier	One-hot encoding and standard scaling
Decision Tree	Tree-based rule model	Raw preprocessed features
SVM with RBF kernel	Maximum-margin nonlinear classifier	One-hot encoding and standard scaling
LightGBM	Gradient boosting with leaf-wise tree growth	Raw preprocessed features
CatBoost	Gradient boosting with ordered boosting and categorical handling	Raw preprocessed features

Table 4: Candidate classification models used in the project.

4.4.1 Logistic Regression

Logistic Regression is a linear probabilistic classifier. It models the log-odds of the positive class as a linear combination of the input features. Although it is relatively simple, it is useful as a baseline model because it is fast, interpretable, and often performs well when the relationship between features and the target is approximately linear.

Because Logistic Regression assumes numerical feature inputs, the categorical variables were handled using one-hot encoding, and the numerical variables were standardized.

4.4.2 Decision Tree

A Decision Tree classifies samples by repeatedly splitting the feature space according to conditions on individual features. It is easy to interpret and can naturally model nonlinear relationships. It also does not require feature scaling.

However, a single decision tree can easily overfit if it grows too deep. Therefore, its depth and splitting parameters need to be controlled during hyperparameter tuning.

4.4.3 Support Vector Machine

The SVM model was trained using an RBF kernel. This allows the model to learn nonlinear decision boundaries by mapping the data into a higher-dimensional feature space. SVMs can perform well on complex classification problems, but they are sensitive to feature scaling and to hyperparameters such as `C` and `gamma`.

For this reason, the SVM model used one-hot encoding for categorical variables and standard scaling for numerical variables.

4.4.4 LightGBM

LightGBM is a gradient boosting method based on decision trees. Unlike standard tree growth methods that expand trees level by level, LightGBM uses a leaf-wise growth strategy, meaning that it expands the leaf that gives the largest loss reduction. This can produce strong models with relatively efficient training.

LightGBM was included as one of the advanced models. It is suitable for tabular data and can also handle class imbalance using parameters such as `scale_pos_weight`. In this project, the ratio between negative and positive samples was approximately:

$$\frac{7260}{2604} \approx 2.79$$

This value was used as a class-imbalance correction parameter for LightGBM.

4.4.5 CatBoost

CatBoost is another gradient boosting model. It uses ordered boosting, which helps reduce target leakage during boosting, and it is known for strong performance on tabular datasets. Although CatBoost can handle categorical features natively, in this project it was applied after the preprocessing stage using the already encoded features.

CatBoost was included as an additional advanced classifier that was not deeply covered in class, allowing us to compare standard models with a stronger modern boosting method.

4.5 10-Fold Stratified Cross-Validation

To obtain a more stable estimate of model performance, we used 10-fold stratified cross-validation. In this method, the training dataset is divided into 10 folds. Each model is trained 10 times, where each fold is used once as a validation fold and the remaining folds are used for training.

Stratification was used so that each fold preserved the original class distribution. This is important because the positive class represents only 26.4% of the data.

The main cross-validation metrics were accuracy, F1-score, and ROC-AUC.

Model	Accuracy Mean	Accuracy Std	F1 Mean	ROC-AUC Mean
Logistic Regression	0.9097	0.0085	0.8234	0.9555
Decision Tree	0.8706	0.0074	0.7571	0.8365
SVM	0.9093	0.0070	0.8217	0.9402
LightGBM	0.9013	0.0062	0.8214	0.9553
CatBoost	0.9112	0.0085	0.8242	0.9580

Table 5: 10-fold stratified cross-validation results for all candidate models.

The cross-validation results show that CatBoost achieved the best overall performance, with the highest accuracy, F1-score, and ROC-AUC. Logistic Regression also performed very strongly, with results close to CatBoost. SVM achieved a similar accuracy and F1-score, but its ROC-AUC was slightly lower. LightGBM also performed well, especially in ROC-AUC, while the Decision Tree had the weakest performance.

The weaker performance of the Decision Tree is expected because a single tree has limited ability to generalize compared to ensemble methods. It can easily overfit specific patterns in the training data or fail to capture smooth decision boundaries. In contrast, CatBoost and LightGBM combine many trees, which usually improves generalization.

4.6 Validation Set Evaluation

After cross-validation, each model was trained on the training split and evaluated on the held-out validation set. This gives another view of model performance on unseen data.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9204	0.8808	0.8081	0.8428	0.9632
Decision Tree	0.8591	0.7288	0.7428	0.7357	0.8218
SVM	0.9123	0.8640	0.7927	0.8268	0.9456
LightGBM	0.9052	0.7982	0.8580	0.8270	0.9634
CatBoost	0.9204	0.8906	0.7965	0.8409	0.9647

Table 6: Validation set performance of all candidate models.

The validation results show that Logistic Regression and CatBoost achieved the highest accuracy, both reaching 0.9204. CatBoost achieved the highest ROC-AUC, with a value of 0.9647, while Logistic Regression was very close with 0.9632. Logistic Regression achieved the highest F1-score, 0.8428, followed closely by CatBoost with 0.8409.

LightGBM achieved the highest recall, 0.8580, meaning that it detected the largest fraction of actual high-intent sessions. However, its precision was lower than Logistic Regression and CatBoost. This means that LightGBM identified more high-intent users, but also produced more false positives.

The Decision Tree again performed worst overall. Although its recall was reasonable, its precision, F1-score, and ROC-AUC were much lower than the other models. This suggests that a single tree is not strong enough for this dataset compared to linear models, SVM, and boosting methods.

4.7 Initial Model Comparison

At this stage, CatBoost and Logistic Regression were the strongest models overall. CatBoost achieved the best ROC-AUC and the best cross-validation performance, while Logistic Regression achieved the best validation F1-score and matched CatBoost in validation accuracy.

SVM also performed well, but did not outperform Logistic Regression or CatBoost. LightGBM was useful because of its high recall, but its lower precision reduced its F1-score. The Decision Tree provided a useful interpretable baseline, but its performance was clearly weaker.

Based on these results, the next steps are to examine confusion matrices and ROC curves, perform hyperparameter tuning, and then select the final model using both performance and statistical comparison.

“latex

4.8 Confusion Matrices

To better understand the type of errors made by each classifier, we examined the confusion matrices on the validation set. The confusion matrix separates the predictions into four cases:

- **True Negative (TN):** A low-intent session correctly predicted as low intent.
- **True Positive (TP):** A high-intent session correctly predicted as high intent.
- **False Positive (FP):** A low-intent session incorrectly predicted as high intent.
- **False Negative (FN):** A high-intent session incorrectly predicted as low intent.

In this project, false negatives are especially important because they represent high-intent users that the model failed to identify.

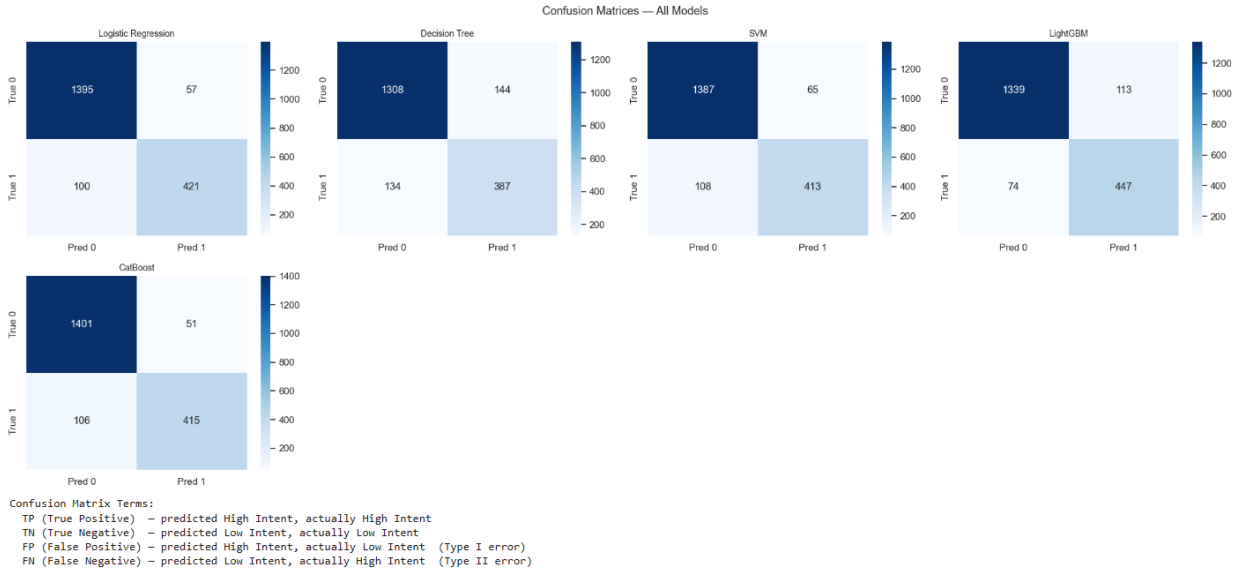


Figure 23: Confusion matrices of all candidate models on the validation set.

The confusion matrices show clear differences between the models. Logistic Regression correctly classified 421 high-intent sessions and missed 100 high-intent sessions. CatBoost had a similar behavior, correctly detecting 415 high-intent sessions and missing 106. SVM detected 413 high-intent sessions and missed 108.

LightGBM detected the largest number of high-intent sessions, with 447 true positives and only 74 false negatives. This explains why LightGBM achieved the highest recall in the validation results. However, it also produced more false positives than Logistic Regression and CatBoost, meaning that it classified more low-intent users as high intent.

The Decision Tree had the weakest confusion matrix overall. It produced more false positives and false negatives than the stronger models, which agrees with its lower validation F1-score and ROC-AUC.

4.9 ROC Curve Analysis

We also compared the models using ROC curves. The ROC curve shows the tradeoff between the true positive rate and the false positive rate at different decision thresholds. A model with a curve closer to the top-left corner performs better, and a larger ROC-AUC value indicates better class separation.

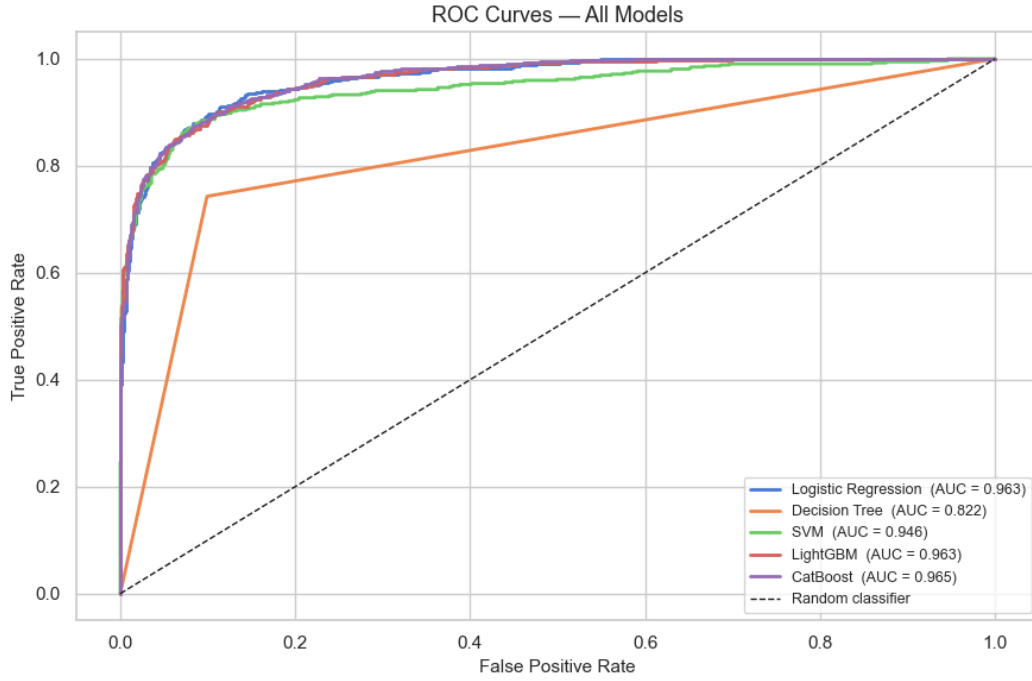


Figure 24: ROC curves of all candidate models on the validation set.

The ROC curves show that CatBoost, LightGBM, and Logistic Regression performed very well and had very similar class-separation ability. CatBoost achieved the highest ROC-AUC, approximately 0.965, followed closely by LightGBM and Logistic Regression with approximately 0.963. SVM also performed well, with ROC-AUC around 0.946, but its curve was slightly lower than the top models. The Decision Tree had the weakest ROC curve, with ROC-AUC around 0.822.

This confirms that the boosting models and Logistic Regression separate high-intent and low-intent sessions much better than a single Decision Tree. It also shows that although LightGBM had more false positives at the default threshold, its probability scores still contain strong ranking information.

4.10 Model Comparison Visualization

To summarize the initial validation performance, we compared the models using accuracy, F1-score, and ROC-AUC.

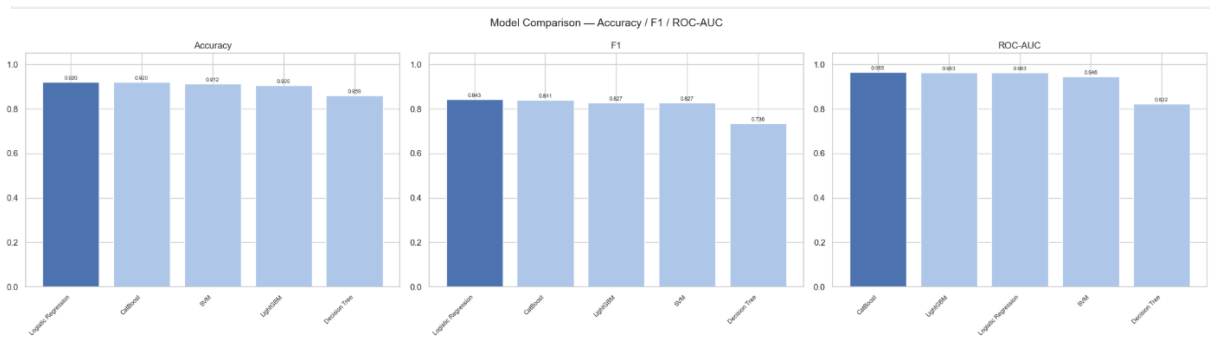


Figure 25: Comparison of validation accuracy, F1-score, and ROC-AUC across all models.

The comparison shows that Logistic Regression, CatBoost, SVM, and LightGBM all performed strongly, while the Decision Tree was clearly weaker. Logistic Regression achieved the highest F1-score, while CatBoost achieved the highest ROC-AUC and matched Logistic Regression in accuracy. LightGBM had slightly lower accuracy and F1-score, but its higher recall makes it useful when the priority is to avoid missing high-intent users.

Overall, the results show that there is no single model that dominates every metric. Logistic Regression is strong and simple, CatBoost has the best class-separation ability, and LightGBM provides the best recall. Therefore, later model selection should consider the business objective and not only raw accuracy.

4.11 Model Behavior Discussion

The different models performed differently because each model has a different structure and learns the decision boundary in a different way.

4.11.1 Logistic Regression

Logistic Regression uses a linear decision boundary. This means that it models the target using weighted combinations of the input features. Even though the real decision boundary is probably not perfectly linear, Logistic Regression still performed very well. This suggests that the preprocessing and the strongest features, especially `page_value`, `engagement_score`, `product_duration`, and abandonment-related features, provide strong separability between the two classes.

Its main advantage is that it is simple, stable, and less likely to overfit. Its weakness is that it may not capture complex nonlinear relationships as well as boosting models.

4.11.2 Decision Tree

The Decision Tree performed worse than the other models. A single tree can easily overfit the training data, especially when the data contains many feature interactions and imbalanced classes. If the tree grows too deep, it may learn very specific rules that do not generalize well. If it is restricted too much, it may underfit.

The weaker performance of the Decision Tree suggests that this dataset requires either smoother decision boundaries or an ensemble of many trees rather than a single tree.

4.11.3 Support Vector Machine

The SVM with an RBF kernel performed well because it can model nonlinear decision boundaries. The RBF kernel allows the classifier to separate the classes in a transformed feature space. However, SVM is sensitive to feature scaling and to hyperparameters such as `C` and `gamma`. It can also be computationally heavier than Logistic Regression and tree-based models.

In this project, SVM achieved competitive results but did not outperform the best models.

4.11.4 LightGBM

LightGBM is an ensemble model that builds many decision trees sequentially. Each new tree focuses on correcting the errors of the previous trees. Its leaf-wise growth strategy can create strong models efficiently and is well suited for tabular datasets.

In our results, LightGBM achieved the highest recall, meaning it found the largest number of high-intent sessions. This is useful when false negatives are expensive. However, it also produced more false positives, which reduced its precision and F1-score.

4.11.5 CatBoost

CatBoost is also a gradient boosting model, but it uses ordered boosting and a symmetric tree structure. These properties help reduce overfitting and improve generalization. CatBoost achieved the best ROC-AUC and the best cross-validation result, showing that it ranked high-intent sessions very well.

Its strong performance suggests that boosting methods are suitable for this dataset, especially because the data contains nonlinear relationships, feature interactions, and categorical information.

4.12 Decision Tree Visualization and Feature Importance

Although the Decision Tree was not the best-performing model, it is useful for interpretation. We visualized a shallow Decision Tree with maximum depth 4 to understand which features appear near the top of the decision process.

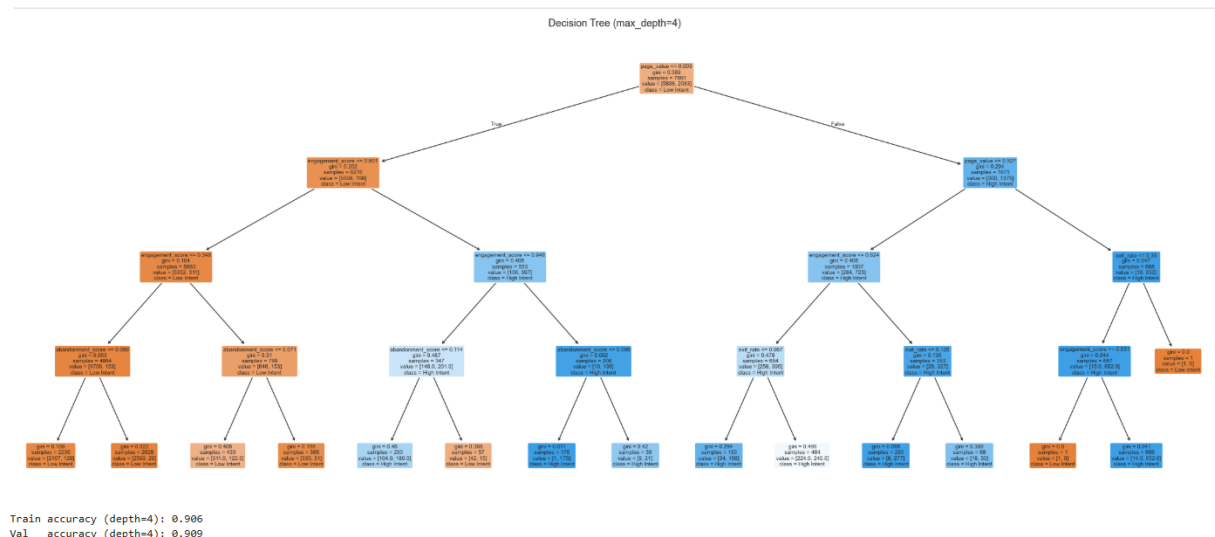


Figure 26: Decision Tree visualization with maximum depth 4.

The tree shows that **page_value** is used near the root of the tree, meaning that it is one of the most important features for separating the classes. This agrees with the exploratory analysis, where **page_value** showed a large difference between low-intent and high-intent sessions.

The shallow tree achieved approximately 0.906 training accuracy and 0.909 validation accuracy. The fact that the validation accuracy is close to the training accuracy suggests that this limited-depth tree is not strongly overfitting. However, its overall performance is still weaker than the stronger models.

We also examined the feature importances from the Decision Tree.

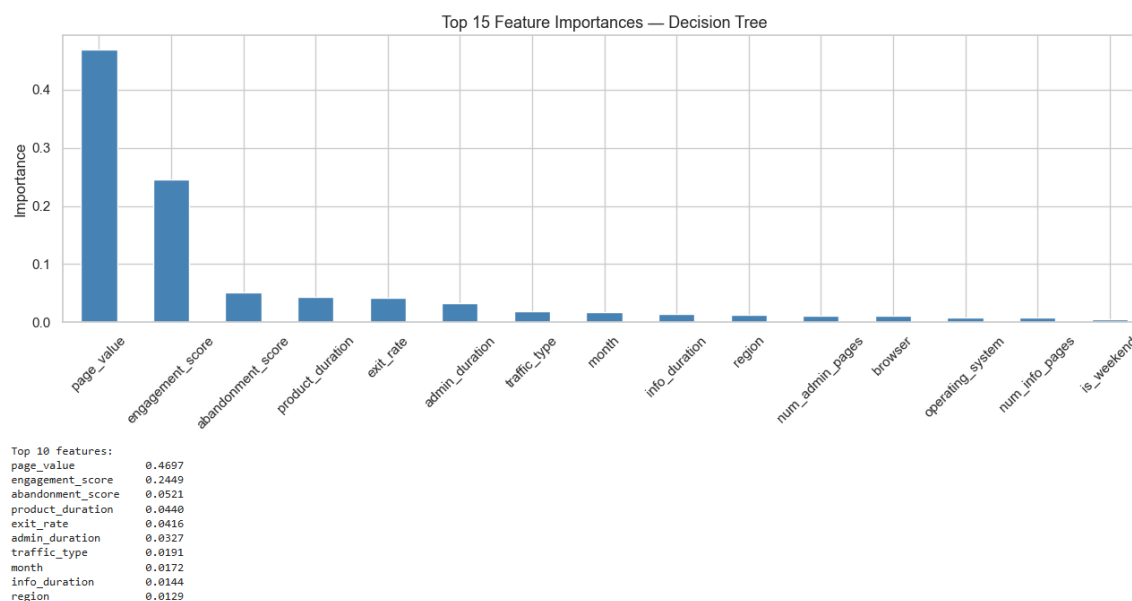


Figure 27: Top feature importances according to the Decision Tree model.

The most important feature was **page_value**, with importance around 0.47. The second most important feature was **engagement_score**, with importance around 0.24. Other useful features included **abandonment_score**, **product_duration**, **exit_rate**, and **admin_duration**.

These results are consistent with the earlier analysis. High-intent users tend to have higher page value, stronger engagement, deeper product browsing, and lower abandonment signals. Therefore, the decision tree interpretation supports the conclusions from the exploratory stage.

4.13 Hyperparameter Tuning

After the initial model comparison, we performed hyperparameter tuning using `GridSearchCV`. The goal was to improve model performance and choose better parameter values for each classifier. We used 5-fold stratified cross-validation during tuning and optimized mainly for F1-score, since F1 balances precision and recall and is appropriate for the moderately imbalanced target variable.

Model	Parameters Searched	Reason
Logistic Regression	C, penalty, solver	Controls regularization strength and helps prevent overfitting.
Decision Tree	max_depth, min_samples_split, criterion	Controls tree complexity and split quality.
SVM	C, kernel, gamma	Controls margin size and nonlinear kernel behavior.
LightGBM	n_estimators, max_depth, learning rate, subsample	Controls boosting complexity, learning speed, and overfitting.
CatBoost	iterations, depth, learning rate	Controls boosting length, tree complexity, and learning speed.

Table 7: Hyperparameters tuned for each model.

The tuning results are summarized below.

Model	CV F1 Before	CV F1 After	Change in F1
SVM	0.8217	0.8275	+0.0058
CatBoost	0.8244	0.8244	+0.0000
Logistic Regression	0.8236	0.8241	+0.0005
LightGBM	0.8167	0.8195	+0.0028
Decision Tree	0.7544	0.8174	+0.0630

Table 8: Hyperparameter tuning summary sorted by tuned F1-score.

The tuning results show that SVM achieved the highest tuned cross-validation F1-score, with a value of 0.8275. CatBoost and Logistic Regression were very close, with F1-scores of 0.8244 and 0.8241. LightGBM improved slightly, while the Decision Tree improved the most.

The large improvement for the Decision Tree is expected because a single tree is highly sensitive to its depth and splitting rules. Before tuning, the tree was more likely to overfit or choose poor split behavior. After tuning, limiting the tree depth and controlling the split parameters improved its generalization.

For Logistic Regression and CatBoost, the improvements were very small. This suggests that their initial settings were already close to optimal for this dataset. SVM improved moderately after tuning, showing that its performance depends more strongly on the choice of kernel and regularization parameters.

Even after tuning, the strongest models remain very close to each other. Therefore, final model selection should consider not only tuned F1-score, but also validation performance, ROC-AUC, recall, stability, and the business cost of false negatives.

“latex

4.14 Tuned Model Validation Results

After hyperparameter tuning, the tuned models were evaluated again on the validation set. The results are shown below.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9214	0.8812	0.8119	0.8452	0.9632
Decision Tree	0.9189	0.8676	0.8177	0.8419	0.9517
CatBoost	0.9204	0.8906	0.7965	0.8409	0.9647
SVM	0.9143	0.8534	0.8157	0.8342	0.9619
LightGBM	0.9057	0.8018	0.8541	0.8271	0.9631

Table 9: Validation performance of the tuned models, sorted by F1-score.

The tuned validation results show that Logistic Regression achieved the highest F1-score, with a value of 0.8452. It also achieved the highest validation accuracy, 0.9214, and a strong ROC-AUC of 0.9632. The tuned Decision Tree improved significantly compared to the initial untuned tree and reached an F1-score of 0.8419, which is very close to Logistic Regression.

CatBoost also remained one of the strongest models. Although its F1-score was slightly lower than Logistic Regression and the tuned Decision Tree, it achieved the highest ROC-AUC, 0.9647. This means that CatBoost separates the classes very well when looking at predicted probabilities across thresholds.

LightGBM achieved the highest recall, 0.8541, meaning that it detected the largest proportion of real high-intent sessions. However, its precision was lower than the other top models, so its F1-score was also lower. This reflects the expected tradeoff: LightGBM catches more high-intent users, but also produces more false positives.

At this stage, Logistic Regression was the provisional best model according to validation F1-score. However, because several models had very close results, we also performed overfitting analysis and statistical significance testing before choosing the final model.

4.15 Overfitting and Underfitting Analysis

To check whether the tuned models were overfitting or underfitting, we compared each model’s training performance with its validation performance. A model is suspected of overfitting if its training score is much higher than its validation score. A model is suspected of underfitting if both scores are low.

The main value we examined was the F1 generalization gap:

$$\text{F1 Gap} = F1_{\text{train}} - F1_{\text{validation}}$$

A large positive gap suggests overfitting. In this project, a gap above 0.10 was treated as a warning sign.

Model	Train Acc.	Val Acc.	Acc. Gap	Train F1	Val F1	F1 Gap
Logistic Regression	0.9091	0.9214	-0.0123	0.8224	0.8452	-0.0228
Decision Tree	0.9115	0.9189	-0.0074	0.8281	0.8419	-0.0138
SVM	0.9096	0.9143	-0.0047	0.8255	0.8342	-0.0086
LightGBM	0.9440	0.9057	0.0383	0.8983	0.8271	0.0711
CatBoost	0.9330	0.9204	0.0125	0.8673	0.8409	0.0264

Table 10: Train-validation comparison for the tuned models.

Figure 28: Training F1, validation F1, and generalization gap for the tuned models.

The results show that none of the models had an F1 gap above the 0.10 warning threshold. Logistic Regression, Decision Tree, and SVM even had slightly higher validation F1 than training F1. This can happen because the validation split may be slightly easier, and the difference is small enough that it does not indicate a problem.

LightGBM had the largest positive gap, with a training F1 of 0.8983 and validation F1 of 0.8271. This suggests that LightGBM fits the training data more strongly than the other models. However, the gap is still below 0.10, so it is not considered severe overfitting. CatBoost had a smaller positive gap of 0.0264, indicating good generalization.

Overall, the overfitting analysis showed that the tuned models generalize reasonably well and that no model clearly underfits or severely overfits.

4.16 Statistical Significance Testing

Since several models had close performance, we used statistical significance testing to support the final model selection. The goal was to check whether the observed differences between models were meaningful or could have happened due to random variation.

First, we collected 10-fold cross-validation F1-scores for the tuned models. The results were:

Model	10-Fold CV F1
Logistic Regression	0.8228 ± 0.0168
Decision Tree	0.8243 ± 0.0098
SVM	0.8264 ± 0.0200
LightGBM	0.8235 ± 0.0115
CatBoost	0.8242 ± 0.0195

Table 11: Tuned 10-fold cross-validation F1-scores.

According to the mean 10-fold CV F1-score, SVM was the best model with a mean F1 of 0.8264. However, the differences between the top models were small, so we performed statistical tests.

The Wilcoxon signed-rank test compared the fold-level F1-scores of SVM against the other models. The results showed that the differences were not statistically significant at the 0.05 level:

Comparison	Wilcoxon Statistic	p-value	Significant?
SVM > Logistic Regression	39.0	0.1377	No
SVM > Decision Tree	30.0	0.4229	No
SVM > LightGBM	33.0	0.3125	No
SVM > CatBoost	44.0	0.0527	No

Table 12: Wilcoxon signed-rank test comparing SVM against the other tuned models.

The Wilcoxon test suggests that SVM had the highest mean CV F1-score, but its improvement over the other models was not statistically significant at the 5% level.

We also used McNemar’s test on the validation set predictions. This test compares pairs of classifiers based on where their predictions disagree. The results showed that some model pairs had statistically significant differences, especially between Logistic Regression and weaker models. However, the comparison between SVM and Logistic Regression was not significant, and the comparison between SVM and CatBoost was also not significant.

Based on the statistical evidence, SVM was selected as the preliminary final individual model because it achieved the highest mean 10-fold CV F1-score and showed stable generalization. However, since the differences between the strongest models were small, we also evaluated imbalance-handling methods and an ensemble approach before making the final submission model decision.

4.17 Imbalanced Data Handling

Because the dataset is moderately imbalanced, we tested additional imbalance-handling strategies. Three methods were considered:

Method	Description
SMOTE	Creates synthetic minority-class samples by interpolating between existing high-intent examples.
ADASYN	Creates more synthetic samples near harder-to-classify minority-class regions.
Random Under-Sampling	Reduces the number of majority-class samples to make the class distribution more balanced.

Table 13: Imbalanced data handling methods tested in the project.

The comparison was performed using the Decision Tree model, because it provides a simple way to observe how resampling changes model behavior.

Metric	Before SMOTE	After SMOTE
Accuracy	0.8591	0.8733
Precision	0.7288	0.7407
Recall	0.7428	0.8004
F1	0.7357	0.7694

Table 14: Decision Tree performance before and after applying SMOTE.

Figure 29: Decision Tree performance before and after SMOTE.

SMOTE improved the Decision Tree performance. The F1-score increased from 0.7357 to 0.7694, and recall increased from 0.7428 to 0.8004. This means that after SMOTE, the model detected more high-intent sessions. The improvement supports the conclusion that class imbalance affects the classification task.

ADASYN also improved recall, reaching 0.8100, with an F1-score of 0.7673. Random under-sampling achieved high recall, 0.8560, but its precision and F1-score were lower. This shows that under-sampling can make the model more sensitive to the positive class, but it may lose too much information from the majority class.

Overall, resampling improved recall, but it did not clearly outperform the strongest tuned models. Therefore, resampling was treated as an additional experiment rather than the main final approach.

4.18 Cost-Sensitive Learning

We also tested cost-sensitive learning. The dataset does not provide explicit monetary costs for false positives and false negatives, so the exact cost ratio is unknown. However, the business logic suggests that false negatives are important because they represent high-intent users that the model fails to detect.

Two cost-sensitive ideas were considered:

1. Assigning higher class weight to the minority class using `class_weight='balanced'`.
2. Adjusting the decision threshold instead of using the default threshold of 0.5.

Lowering the decision threshold usually increases recall, because more users are classified as high intent. However, this often reduces precision, because more low-intent users are also classified as high intent.

In this experiment, different thresholds were tested for a class-weighted Decision Tree. The results were almost identical across the tested thresholds:

Threshold	Precision	Recall	F1	Accuracy
0.3	0.7559	0.7370	0.7464	0.8677
0.4	0.7559	0.7370	0.7464	0.8677
0.5	0.7559	0.7370	0.7464	0.8677
0.6	0.7559	0.7370	0.7464	0.8677

Table 15: Cost-sensitive threshold analysis using a class-weighted Decision Tree.

The similar results across thresholds suggest that the tree produced probability outputs that did not change the final classifications much in this range. Therefore, this approach did not provide a strong improvement. Still, it is useful conceptually because if the business later provides real costs for missed high-intent users or wasted marketing actions, these costs can be incorporated through class weights or threshold adjustment.

4.19 Precision-Recall Curves

Since the positive class is the more important and less frequent class, we also examined precision-recall curves. Precision-recall analysis is especially useful when the dataset is imbalanced because it focuses directly on positive-class performance.

Figure 30: Precision-recall curves for all tuned models.

The precision-recall curves show that the strongest models maintain high precision over a wide range of recall values. CatBoost and LightGBM achieved the highest average precision, both around 0.927. Logistic Regression was also strong, with average precision around 0.923. SVM was slightly lower, with average precision around 0.913.

The Decision Tree curve is less informative because its probability outputs are less smooth and less calibrated than the other models. This again shows that although the tuned Decision Tree improved, the stronger models provide better ranking and probability behavior.

4.20 Final Individual Model Evaluation

Based on the 10-fold CV F1-score and statistical comparison, SVM was selected as the final individual model before ensemble testing. Its validation classification report is summarized below.

Class	Precision	Recall	F1-score	Support
Low Intent (0)	0.93	0.95	0.94	1452
High Intent (1)	0.85	0.82	0.83	521
Accuracy		0.91		1973
Macro Avg.	0.89	0.88	0.89	1973
Weighted Avg.	0.91	0.91	0.91	1973

Table 16: Classification report for the selected SVM model on the validation set.

Figure 31: Confusion matrix and ROC curve for the selected SVM model.

The SVM model achieved strong validation performance. It correctly classified 425 high-intent sessions and missed 96 high-intent sessions. Its ROC-AUC was approximately 0.962, showing that it separates low-intent and high-intent sessions well.

The final individual SVM model was then used to generate a test prediction file. The resulting prediction distribution on the test set was:

$$\text{Predicted Low Intent} = 1874$$

$$\text{Predicted High Intent} = 592$$

This corresponds to a predicted high-intent rate of approximately:

$$\frac{592}{2466} \approx 24.0\%$$

This is close to the training positive rate of 26.4%, suggesting that the model's predictions are reasonable and not heavily biased toward either class.

4.21 Voting Ensemble and Final Model Selection

As a final experiment, we tested a soft voting ensemble using the tuned models. A soft voting ensemble averages the predicted probabilities of several models and then applies a classification threshold. This can improve performance because different models may make different errors.

The ensemble included five models:

Logistic Regression, Decision Tree, SVM, LightGBM, CatBoost

Several decision thresholds were tested, from 0.35 to 0.65. The best threshold was selected based on validation performance.

Figure 32: Voting ensemble accuracy and F1-score as a function of the decision threshold.

The best ensemble threshold was 0.55. At this threshold, the voting ensemble achieved:

Accuracy = 0.9265

F1-score = 0.8546

Recall = 0.8177

Precision = 0.8950

ROC-AUC = 0.9659

The best individual model by validation accuracy was Logistic Regression, with accuracy 0.9214. The voting ensemble achieved a higher validation accuracy of 0.9265, and it also achieved a higher F1-score than the individual models. Therefore, the voting ensemble was selected as the final model for prediction generation.

The final selected model was:

Soft Voting Ensemble with threshold 0.55
--

This model was chosen because it provided the best overall validation performance while combining the strengths of several different classifiers. Logistic Regression provides a stable linear baseline, SVM captures nonlinear decision boundaries, and the boosting models capture feature interactions. By averaging their predicted probabilities, the ensemble reduced dependence on a single model and improved the final validation result. “

5 Final Summary